Ecole Universitaire de physique et d'ingénierie

Projet de synthese M2 PAR
Rapport de projet

Vision neuromorphique pour la navigation d'un robot mobile

Élèves :
Nadjib

Enseignant :
Encadrant a completer

16 février 2026

Table des matières

Introduction **1**

1 Materiel et fondements theoriques **2**
1.1 Presentation du materiel 2
1.1.1 Presentation de la camera DAVIS346 et fiche technique 2
1.1.2 Presentation du robot LIMO 3
1.2 Concepts theoriques mobilises 3
1.2.1 Camera a evenements : notion, bref historique, avantages et limites 3
1.2.2 Detection d'obstacles (IMO) : panorama rapide des familles 3
1.2.3 Navigation autonome et algorithme utilise dans le projet 4

2 Methodologie **5**
2.1 Motion compensation 5
2.2 Objects segmentation 5
2.3 Objects clustering 6
2.4 Creation d'une layer Nav2 pour les obstacles dynamiques 6

4 Tests et analyse des resultats **7**
4.1 Protocole experimental 7
4.2 Resultats 7
4.3 Analyse 7
4.4 Discussion 8

Conclusion **9**

Table des figures

Liste des tableaux

1.1 Fiche technique synthetique de la DAVIS346 2

Introduction

Ce rapport presente la conception et l'integration d'une chaine de perception neuromorphique pour la navigation autonome d'un robot mobile LIMO equipe d'une camera DAVIS346.

Le coeur du projet est l'implementation d'une methode de detection d'objets en mouvement basee sur la compensation d'ego-mouvement par IMU dans un flux d'evenements. Cette approche est ensuite raccordee a Nav2 afin d'injecter des obstacles dynamiques dans la prise de decision de navigation.

Le document est organise comme suit :

— le chapitre 1 introduit le materiel et les fondements theoriques ;
— le chapitre 2 detaille la methodologie d'implementation de l'article retenu et son adaptation ROS 2 ;
— le chapitre 4 presente le protocole de test et l'analyse des resultats ;
— la conclusion synthese les acquis et les perspectives.

Chapitre 1

Materiel et fondements theoriques

1.1 Presentation du materiel

1.1.1 Presentation de la camera DAVIS346 et fiche technique

La camera DAVIS346 est un capteur neuromorphique hybride qui produit en parallele un flux d'evenements (DVS) et des images APS classiques sur un meme capteur. Cette architecture est interessante pour notre projet, car elle permet de conserver une information temporelle tres fine tout en gardant une reference image exploitable pour le debug et la visualisation [1].

Les caracteristiques techniques directement utiles pour la chaine de perception du projet sont resumees dans le tableau 1.1.

Table 1.1 – *Fiche technique synthetique de la DAVIS346*

Parametre	Valeur
Resolution capteur	346 x 260
Sorties	Evenements DVS + images APS concurrentes
Plage dynamique evenementielle	Jusqu'a 120 dB
Resolution temporelle	1 us
Latence evenementielle	Sous 1 us (nominale)
Debit maximal	Jusqu'a 12 MEPS
IMU integree	6 axes (gyro + accelerometre), jusqu'a 8 kHz
Alimentation	$<$ 180 mA a 5 V
Connectique principale	USB 3.0 Micro-B (verrouillable)
Monture optique	CS-mount (adaptateur C-mount possible)

Le document technique signale egalement des limites pratiques a connaitre lors des essais : salves d'evenements possibles en mode APS Global Shutter, schema de balayage DVS sous forte charge, et performances image APS inferieures a des cameras classiques. Ces limites justifient un reglage fin des biais capteur et une calibration experimentale adaptee au contexte robotique [1].

1.1.2 Presentation du robot LIMO

Le robot LIMO d'AgileX est presente comme une plateforme ROS polyvalente avec quatre modes de locomotion (differentiel, Ackermann, chenilles et mecanum), ce qui en fait un support pertinent pour des tests de navigation dans des scenes heterogenes [2, 3].

La documentation technique indique en particulier :

— dimensions compactes (322 x 220 x 251 mm) et charge utile de 4 kg ;
— vitesse maximale sans charge de l'ordre de 1 m/s ;
— calcul embarque base sur NUC sous Ubuntu 22.04 ;
— capteurs integres utiles a la navigation (LiDAR, camera de profondeur, IMU).

La pile logicielle officielle propose des launches ROS 2 pour le demarrage de la base, la cartographie, puis la navigation Nav2 suivant le mode de conduite retendu [2]. Cette base materielle et logicielle est compatible avec notre integration de la perception evenementielle.

1.2 Concepts theoriques mobilises

1.2.1 Camera a evenements : notion, bref historique, avantages et limites

Une camera a evenements est un capteur asynchrone : chaque pixel declenche un evenement uniquement lors d'une variation de luminance, au lieu d'attendre une image globale echantillonnee a frequence fixe. La revue de reference rappelle que ce paradigme s'inscrit dans le champ plus large de l'ingenierie neuromorphique et qu'il s'est structure au cours des dernieres decennies avec des familles de capteurs comme DVS ou ATIS [4].

Les avantages majeurs pour la robotique mobile sont :

— forte plage dynamique (ordre de 120 dB) utile en eclairage difficile ;
— tres faible latence de perception ;
— reduction de la redondance et de la consommation grace au codage asynchrone des changements [4].

Les limites relevees dans la meme revue sont aussi importantes :

— flux sparse plus difficile a exploiter que des images denses ;
— capteurs encore couteux ;
— resolution et interfaces encore contraintes sur certains systemes commerciaux [4].

1.2.2 Detection d'obstacles (IMO) : panorama rapide des familles

La detection d'objets en mouvement independant (IMO) vise a separer les mouvements lies a l'ego-mouvement du robot de ceux des objets reels de la scene. Dans un cadre evenementiel, la revue distingue trois familles methodologiques principales [4] :

— approches de compensation de mouvement (soustraction du mouvement global, souvent via flot optique) ;
— approches residuelles iteratives combinant ego-mouvement et segmentation de residus ;
— approches apprentissage (supervise ou non supervise) pour segmenter directement les IMO.

Ce panorama montre que les approches a base de compensation restent attractives pour des contraintes de temps reel embarque, ce qui motive le choix methodologique du projet.

1.2.3 Navigation autonome et algorithme utilise dans le projet

La documentation LIMO rappelle le schema classique de navigation mobile : localisation sur carte + planification globale + planification locale [2]. En pratique, l'execution de la navigation est faite via Nav2 avec un lancement dedie au mode differentiel (`limo_nav2_diff.launch.py`) [2].

Dans notre configuration, le planificateur global est de type NavFn (equivalent Dijkstra quand l'option A* est inactive), le controle local repose sur DWB, et la localisation est assuree par AMCL. Cette architecture sert de base pour injecter ensuite les obstacles dynamiques issus de la perception evenementielle.

Chapitre 2

Methodologie

Ce chapitre detaille l'implementation de l'article *Event-based Real-time Moving Object Detection Based On IMU Ego-motion Compensation* dans notre environnement ROS 2 [5].

2.1 Motion compensation

L'etape cle est la compensation de l'ego-mouvement de la camera. Dans l'article, les auteurs associent des paquets d'evenements a des mesures IMU sur une fenetre temporelle courte, puis appliquent une fonction de warping non lineaire issue d'un modele de rotation rigide. L'objectif est de realigner les evenements du fond statique pour rendre les zones dynamiques plus visibles [5].

Toujours selon l'article, la formulation non lineaire ameliore la precision de compensation (ordre de 10 a 15%) par rapport a une approximation lineaire, tout en restant compatible avec une execution temps reel embarquee [5].

Sur le plan logiciel, nous sommes partis de la version ROS1 du package de motion compensation puis nous avons realise une adaptation ROS 2 dans le package `datasync_3_0`. Le noeud ROS 2 :

- subscribe les evenements et l'IMU ;
- calcule une vitesse angulaire moyenne sur fenetre ;
- produit des evenements compenses, une count-image, une time-image et un seuil dynamique λ publie sur un topic dedie.

2.2 Objects segmentation

Apres compensation, le papier exploite le fait que les evenements de fond stabilises se concentrent spatialement, alors que les evenements d'objets independants restent plus disperses. La segmentation est faite via une count-image et une time-image : un pixel est classe dynamique si sa valeur temporelle normalisee depasse un seuil dynamique $\lambda = a\|\omega\| + b$ [5].

Dans notre implementation ROS 2 (`project_ws/src/event_segmentation`), le noeud de segmentation :

- synchronise `/time_image` et `/count_image` ;
- recupere λ depuis `/event_lambda_threshold` ;

— genere un masque binaire des pixels foreground (avec contraintes de minimum d'activite et filtrage morphologique optionnel).

2.3 Objects clustering

La derniere etape de l'article regroupe les evenements dynamiques en objets. La logique theorique combine un clustering type DBSCAN (robuste au bruit) avec une metrique melant position spatio-temporelle 3D et flot optique 2D pour mieux separer des objets proches dans le plan image [5].

Dans notre chaine ROS 2 (`project_ws/src/event_clustering`), la version actuelle realise un regroupement en composantes connexes du masque foreground, complete par un filtrage temporel de persistance des petites composantes. Cette adaptation conserve l'esprit de la methode (regroupement spatio-temporel robuste en temps reel) tout en restant simple a maintenir dans le pipeline global.

2.4 Creation d'une layer Nav2 pour les obstacles dynamiques

Les tutoriels officiels Nav2 decrivent deux voies complementaires pour injecter les obstacles dynamiques dans la navigation :

— creer un plugin de costmap en derivant `nav2_costmap_2d::Layer` et en implementant les methodes `onInitialize()`, `updateBounds()` et `updateCosts()` ;

— exporter ce plugin via `PLUGINLIB_EXPORT_CLASS` et l'activer dans la liste `plugins` de la costmap [6].

La documentation de configuration Nav2 rappelle aussi que les couches de costmap peuvent ingerer des sources capteurs via `observation_sources`, notamment en `LaserScan` ou `PointCloud2` [7].

Dans notre projet, l'integration operationnelle est faite par cette seconde voie : publication des obstacles dynamiques au format `PointCloud2` sur `/dynamic_obstacles`, puis declaration de cette source dans les costmaps locale et globale (fichier `nav2_diff.yaml`).

Chapitre 4

Tests et analyse des resultats

4.1 Protocole experimental

L'evaluation est structuree en quatre blocs, alignes sur la chaine de traitement complete :

- tests de compensation de mouvement (stabilite du fond apres warping) ;
- tests de segmentation (qualite du masque foreground) ;
- tests de clustering (coherence des objets detectes) ;
- tests d'integration navigation (prise en compte des obstacles dynamiques dans Nav2).

Les scenarios incluent des phases statiques, des rotations du robot, puis des deplacements avec un ou plusieurs objets en mouvement. Les indicateurs suivis restent volontairement simples :

- densite visuelle des evenements compenses sur le fond ;
- taux de bruit residuel dans le masque ;
- stabilite des contours d'objets sur des sequences consecutives ;
- impact sur la trajectoire Nav2 (detours, arrets de securite, absence de collision).

4.2 Resultats

Les essais fonctionnels confirment le comportement attendu de la pipeline :

- la compensation IMU reduit la dispersion des evenements de fond et rend les zones dynamiques plus discriminantes ;
- la segmentation par seuil dynamique produit un masque exploitable meme quand la scene contient des changements rapides ;
- le clustering permet de convertir le masque en objets utilisables pour la navigation.

Sur le volet navigation, l'ajout de la source `/dynamic_obstacles` dans les costmaps locale et globale permet de refleter les obstacles detectes dans la carte de cout et d'influencer le plan local en temps reel.

4.3 Analyse

L'analyse met en evidence trois points :

— le couplage IMU + evenement est essentiel : sans compensation, la segmentation degrade rapidement des que l'ego-mouvement augmente ;
— le seuil dynamique λ est un parametre sensible qui conditionne le compromis entre bruit et rappel d'objets ;
— l'integration Nav2 via les costmaps est robuste tant que le topic `/dynamic_obstacles` est publie de facon stable.

Les limites actuelles concernent surtout les cas de faible texture, les objets tres proches les uns des autres, et les scenes a fort bruit evenementiel.

4.4 Discussion

La solution retenue est coherente avec l'objectif du projet : obtenir une perception d'obstacles dynamiques compatible avec des contraintes temps reel sur robot mobile.

Les pistes d'amelioration prioritaires sont :
— calibration plus fine des parametres de compensation et de segmentation selon les conditions lumineuses ;
— enrichissement du clustering (metriques mouvement plus explicites) ;
— etape supplementaire d'association multi-frames pour stabiliser les objets et estimer leur vitesse avant projection dans Nav2.

Conclusion

Ce rapport a presente une chaine complete allant de la perception evenementielle a l'integration navigation sur robot mobile. Le materiel (DAVIS346 + LIMO), les fondements theoriques, puis les choix d'implementation ROS 2 ont ete explicites de maniere coherente avec l'article de reference.

La contribution principale est l'adaptation d'une approche de detection d'objets dynamiques basee sur compensation IMU, segmentation temporelle et clustering, puis son raccordement a Nav2 via l'injection d'obstacles dynamiques dans les costmaps.

Les suites naturelles du travail sont :

— renforcer la robustesse en scenes fortement dynamiques et peu texturees ;
— raffiner le suivi temporel des objets detectes ;
— consolider l'evaluation quantitative (latence, precision detection, impact navigation) sur des campagnes de tests plus longues.

Bibliographie

[1] iniVation AG. *DAVIS346 Datasheet*, 2025. Release 2025-08-05, technical documentation.

[2] AgileX Robotics. Limo ros2 humble documentation. `https://raw.githubusercontent.com/agilexrobotics/limo_ros2_doc/master/LIMO-ROS2-humble%28EN%29.md`, 2026. Accessed on 2026-02-16.

[3] AgileX Robotics. Limo, the first multi-modal mobile robot with ai modules and ros demo. `https://www.youtube.com/watch?v=G19B7R4gIGU`, 2026. Video presentation, accessed on 2026-02-16.

[4] A. Novoa, F. Lobon, H. G. De Marina, S. Romero, and F. Barranco. Neuromorphic perception and navigation for mobile robots : A review. *arXiv preprint arXiv :2407.06792*, 2024.

[5] Chunhui Zhao, Yakun Li, and Yang Lyu. Event-based real-time moving object detection based on imu ego-motion compensation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 690–696, 2023.

[6] Nav2 Documentation. Writing a new costmap2d plugin. `https://docs.nav2.org/plugin_tutorials/docs/writing_new_costmap2d_plugin.html`, 2026. Accessed on 2026-02-16.

[7] Nav2 Documentation. Configuring costmaps. `https://docs.nav2.org/configuration/packages/configuring-costmaps.html`, 2026. Accessed on 2026-02-16.